

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ВІДОКРЕМЛЕНИЙ СТРУКТУРНИЙ ПІДРОЗДІЛ
«ПОЛТАВСЬКИЙ ПОЛІТЕХНІЧНИЙ ФАХОВИЙ КОЛЕДЖ
НАЦІОНАЛЬНОГО ТЕХНІЧНОГО УНІВЕРСИТЕТУ
«ХАРКІВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ»

Циклова комісія дисциплін програмної інженерії

КУРСОВА РОБОТА

на тему _____ «Розробка мобільної гри»

на тему **«Розробка мобільної 2D-гри "Bike Evolution"»**

Виконав: здобувач освіти 2 курсу, групи
24 спеціальності 121 «Інженерія програмного
забезпечення» Кеда М.О.

Керівник

Олійник В.В

(підпис)

(прізвище та ініціали)

Полтава – 2026

ЗМІСТ

ВСТУП.....	3
1. ПОСТАНОВКА ЗАВДАННЯ.....	4
1.1. Основні вимоги до продукту.....	4
1.2. Вимоги до інтерфейсу.....	5
2. ПЛАНУВАННЯ СИСТЕМИ.....	7
2.1. Алгоритм роботи програми.....	7
2.2. Тестування.....	9
2.3. Інструкція з використання системи.....	9
ВИСНОВКИ.....	10
СПИСОК ВИКОРИСТАНОЇ ЛІТЕРАТУРИ.....	11
ДОДАТОК А. ПОРІВНЯЛЬНА ХАРАКТЕРИСТИКА ПРОГРАМНИХ ПРОДУКТІВ АНАЛОГІЧНОГО ПРИЗНАЧЕННЯ.....	12
ДОДАТОК Б. UML ДІАГРАМА ПРЕЦЕДЕНТІВ.....	13
ДОДАТОК В. ПРОТОТИП ІНТЕРФЕЙСУ.....	14
ДОДАТОК Г. UML ДІАГРАМА КЛАСІВ.....	15
ДОДАТОК Д. UML ДІАГРАМА РОЗГОРТАННЯ.....	16
ДОДАТОК Е. ВИХІДНІ КОДИ.....	17
ДОДАТОК Ж. РЕЗУЛЬТАТИ ТЕСТУВАННЯ.....	20
ДОДАТОК И. ЗНІМКИ ЕКРАНУ.....	21

ВСТУП

Під час мого навчання на другому курсі я здобув базові теоретичні та практичні знання з багатьох профільних дисциплін. Це дало мені змогу суттєво поглибити знання у сфері архітектури розробки програмного забезпечення, а також вдосконалити навички написання чистого та оптимізованого коду. В якості завдання для курсової роботи переді мною була поставлена задача розробити мобільну гру. Для створення гри я вибрав сучасний ігровий рушій Unity та об'єктно-орієнтовану мову програмування C#. Для написання коду я використовував середовище Visual Studio Code, яке забезпечує зручність розробки, швидке автодоповнення та легкість налагодження коду.

Сучасна індустрія мобільних розваг розвивається швидкими темпами, і жанр 2D-симуляторів водіння з елементами фізичного балансування залишається одним із найбільш затребуваних серед користувачів мобільних пристроїв. Це пов'язано з тим, що подібні ігри пропонують простий для розуміння, але водночас глибокий геймплей. Геймплей вимагає від гравця концентрації, аналізу поведінки транспортного засобу та прийняття рішень щодо покращення окремих вузлів техніки в гаражі. Розробка такого проекту потребує розуміння базових засад класичної механіки, методів обробки подій вводу на мобільних пристроях, а також побудови ефективних систем збереження прогресу.

Актуальність теми курсової роботи полягає в розробці програмного продукту, що поєднує в собі фізичний симулятор та економічну модель покращень. Створення 2D-гри про мопеди з можливістю тюнінгу дозволяє не лише реалізувати ігрові потреби користувачів, а й продемонструвати навички програмування складних взаємопов'язаних систем. Гравець повинен самостійно приймати рішення, які запчастини купувати та як це вплине на швидкість мопеда і його здатність долати перешкоди.

Метою роботи є розробка мобільної 2D-гри «Bike Evolution», яка надасть користувачам можливість кататися на мопеді по пересіченій місцевості, заробляти монети за проходження чекпоінтів та виконання трюків, а також покращувати технічні характеристики двигуна та зірок трансмісії безпосередньо в меню гаражу.

Для досягнення поставленої мети необхідно виконати такі завдання:

—розробити фізичну модель руху двоколісного транспорту з урахуванням обертів двигуна (RPM), крутного моменту, передавального відношення трансмісії та аеродинамічного опору;—реалізувати чутливу систему керування вводом, адаптовану для мобільних пристроїв (сенсорні кнопки газу, гальма та трюків);—розробити інтуїтивно зрозумілий графічний інтерфейс користувача з використанням Unity UI (uGUI), включаючи вікна

головного меню, гаражу тюнінгу та ігрового процесу;—створити логіку покращення двигуна та трансмісії з автоматичним розрахунком прогресивної вартості рівнів;—реалізувати надійну систему локального збереження даних гравця (балансу та рівнів деталей) за допомогою серіалізації в JSON-файл;—провести тестування розроблених систем, виявити та усунути помилки в алгоритмах руху та збереження.Предметом дослідження є розробка мобільної гри в середовищі Unity. Цей проект передбачає створення програмного продукту, який дозволить користувачам грати в зручному графічному інтерфейсі в офлайн-режимі.

Методи дослідження: в процесі виконання курсової роботи були використані наступні методи:

- Теоретичні — проведення порівняльного аналізу існуючих офлайн-симуляторів водіння з метою вивчення їх структури та геймплею.
- Системний аналіз — детальний розгляд і розподіл гри на складові частини, такі як фізика руху, механіка тюнінгу, локальне збереження даних тощо. Це дозволить зрозуміти, як різні елементи гри взаємодіють між собою і як можна оптимізувати їх роботу.

1. ПОСТАНОВКА ЗАВДАННЯ

В невеликих містечках, багато підлітків захоплюються мототехнікою. Вони проводять години в гаражах, вивчаючи будову двигунів та покращуючи популярні мопеди, такі як "Дельта". Проте не у всіх ентузіастів є доступ до власного гаража, інструментів чи коштів на купівлю таких запчастин, як нова циліндро-поршнева група. Ця ситуація стала початком ідеї перенести гаражну романтику у віртуальний світ.

Було вирішено створити власну мобільну гру "Bike Evolution". Я взявся за написання програмного коду, використовуючи середовище розробки Visual Studio Code, вибравши мову програмування C# та двигун Unity. Програмний продукт створювався з акцентом на мобільні платформи для гри без необхідності підключення до інтернету.

У грі користувачі отримують базовий мопед і можуть переміщатися 2D-світом, заробляючи ресурси на купівлю нових запчастин. Виграє той підхід, який дозволяє гравцю створити найшвидший та найкраще налаштований транспортний засіб. Ця гра стала не лише засобом розваги, а й майданчиком для реалізації технічних інтересів.

1.1. Основні вимоги до продукту

Гра повинна бути написана на мові програмування C#.

Функціональні вимоги визначають конкретні дії та операції, які програмний продукт повинен виконувати під час роботи з користувачем. Для гри «Bike Evolution» було сформульовано такий список функціональних вимог:

—Фізичне моделювання руху: Гра повинна обробляти фізику двоколісного мопеда на базі Rigidbody2D. Необхідно розраховувати кутові швидкості коліс, зчеплення з поверхнею траси, а також аеродинамічний опір та опір коченню коліс. Фізичний рух повинен реагувати на натискання кнопок газу та гальм.—Система трюків та балансування (Stunts): Гра повинна мати механіку їзди на задньому колесі (wheelie). Активація кнопки трюку повинна створювати додатковий обертальний момент на раму мопеда для підняття переднього колеса, а система допомоги Wheelie Assist повинна запобігати швидкому падінню.—Гараж та покращення (Upgrade System): Користувач

повинен мати доступ до інтерфейсу покращення двигуна та зірок трансмісії. Купівля покращень повинна списувати монети відповідно до формули вартості та одразу ж змінювати параметри максимальної потужності (maxHP) двигуна та передавального відношення.—Локальне збереження даних: Гра має зберігати поточний стан балансу монет та рівні деталей на мобільному пристрої у файлі `playerdata.json`, який завантажується автоматично при старті гри.—Температурний режим роботи двигуна: При тривалому русі на високих обертах двигун повинен нагріватися. Досягнення критичної температури призводить до перегріву, вимкнення двигуна та програшу заїзду. Проходження чекпоінтів повинно скидати температуру до безпечних значень. Нефункціональні вимоги виражають системні обмеження та характеристики якості програмного забезпечення:

—Продуктивність та оптимізація: Гра повинна працювати стабільно з частотою кадрів не менше 60 FPS на мобільних пристроях середнього сегменту, не перевантажуючи оперативну пам'ять та процесор.—Автономність: Програма повинна бути повністю автономною, працюючи в режимі офлайн без необхідності підключення до мережі Інтернет для перевірки ліцензії чи завантаження конфігурацій.—Надійність та збереження даних: Будь-які аварійні виходи з програми чи падіння рівня не повинні призводити до пошкодження файлу збереження. Зароблені монети мають записуватися надійно. ПЕРЕЛІК НЕРЕАЛІЗОВАНОЇ ФУНКЦІОНАЛЬНОСТІ

- не реалізовано ведення розширеної статистики ігор;
- не реалізовано систему досягнень.
- збереження в хмарі.

Перелік нереалізованих функцій пов'язаний з недостатнім рівнем знань та складністю їх впровадження. Відсутність певних навичок у конкретних

областях програмування може створювати перешкоди для реалізації певних задач.

1.2. Вимоги до інтерфейсу

Вимоги користувача стосовно зовнішнього вигляду програми полягають у створенні мінімалістичного, не перевантаженого текстурами дизайну. Навігація має бути інтуїтивно зрозумілою: елементи керування в грі не повинні перекривати огляд траси, а в меню тюнінгу гравець повинен чітко бачити характеристики (поточні та майбутні).

Ці вимоги візуалізовано за допомогою прототипу інтерфейсу, створеного з застосуванням сервісу canva.com.

В ході створення продукту було використано GUI-фреймворк Unity UI (uGUI), який є стандартизованим візуальним конструктором всередині редактора Unity.

ПРИЙНЯТІ РІШЕННЯ В ПЛАНІ ПОЛІПШЕННЯ ЗРУЧНОСТІ КОРИСТУВАННЯ (UX):

- Кнопки газу та гальма розміщені в нижніх правому та лівому кутах екрана відповідно, що ідеально підходить для хвату смартфона двома руками.
- Розмір кнопок меню збільшено для уникнення хибних натискань на екранах невеликої діагоналі.
- Вигляд кнопок має мінімалістичний стиль, та невелику кольорову гамму.
- Кількість тексту на екрані зменшена по мінімуму, а іконок і зображень більше.

Зовнішній вигляд програми складається з кількох ключових вікон. Головне меню містить великий логотип та кнопки навігації ("Грати", "Гараж", "Вихід"). Вікно гаража розділене на дві зони: по центру відображається сам мопед, а в нижній або боковій частині екрана розташоване меню з категоріями деталей та їх вартістю. Ігровий екран (сцена заїзду) позбавлений зайвих

елементів, залишаючи лише лічильник швидкості, баланс монет і напівпрозорі сенсорні кнопки керування.

2. ПЛАНУВАННЯ СИСТЕМИ

2.1. Алгоритм роботи програми

Для забезпечення стабільної роботи та зручності користувача мобільна гра «Bike Evolution» функціонує за чітко визначеним ігровим циклом, який включає ініціалізацію, основний цикл фізичного моделювання, інтерфейс тюнінгу в гаражі та систему збереження прогресу.

Ігровий цикл побудований на наступних етапах:

—Завантаження даних: При запуску гри зчитуються збережені значення монет (Coins), рівня двигуна (EngineLevel) та рівня головної передачі (PrimaryDriveLevel). Якщо збереження відсутні, гра починається з базовими характеристиками першого рівня.—Головне меню та Гараж: Користувач може вибрати запуск заїзду або перехід до гаражу для покращення деталей. Модифікація деталей двигуна підвищує максимальну потужність, а зміна передавального числа зірок дозволяє оптимізувати передачу крутного моменту для досягнення більшої максимальної швидкості. Уся робота гри підпорядкована строгому циклічному алгоритму, який починається з ініціалізації середовища Unity та завантаження локального профайлу гравця, після чого починається основний ігровий цикл фізичного оновлення.

1. Ініціалізація та Завантаження: При запуску сцени «Login 1» створюється екземпляр класу SaveRepository (клас OfflineSaveSystem). Цей сервіс перевіряє наявність файлу playerdata.json у директорії Application.persistentDataPath. Якщо файл існує, його вміст зчитується за допомогою методу File.ReadAllText та десеріалізується в об'єкт даних. У разі відсутності файлу ініціалізується новий екземпляр з нульовим балансом монет та першим рівнем деталей двигуна.

2. Цикл Гаражу та Тюнінгу: Гравець переходить у вікно гаражу, де клас UpgradeUI зчитує поточні рівні покращень та оновлює текстові поля. При натисканні кнопки покупки викликається метод UpgradeEngine чи

UpgradePrimaryDrive. Метод GetUpgradeCost розраховує вартість за формулою: $Cost = 2 * Lvl * Lvl + 16 * Lvl + 32$. Якщо $Coins \geq Cost$, баланс монет зменшується, рівень деталі підвищується на 1, викликається PlayerPrefs.Save() та оновлюються параметри мопеда в RaceService за допомогою методу ApplyUpgrades.

3. Фізичний ігровий цикл (FixedUpdate): Після запуску заїзду ігровий двигун Unity викликає метод FixedUpdate класу BikeController кожні 0.02 секунди для розрахунку фізики. Метод обробляє поточний стан дроселя (throttleInput) та застосовує крутний момент до заднього колеса (rearWheelRb.AddTorque). При активації кнопки Stunt рама отримує додатковий крутний момент (stuntPullTorque).

4. Система автоматичної трансмісії (АКПП): Клас BikeController реалізує автоматичне перемикання передач. Якщо RPM перевищує shiftUpRPM (7500 об/хв), передача підвищується. Якщо оберти падають нижче shiftDownRPM (3000 об/хв), передача знижується. При активованому трюку перемикання передач блокується для запобігання втрати стабільності балансу.

5. Термодинамічна модель двигуна: Температура двигуна розраховується за формулою: при високих обертах температура зростає зі швидкістю $baseHeatRate * engineHeatMultiplier * Time.fixedDeltaTime$. На холостому ході або при русі накатом температура знижується відповідно до ambientCoolingRate. При досягненні 120°C двигун глохне.

6. Збереження прогресу при виході: Під час натискання клавіші ESC або при фінішуванні заїзду, накопичена валюта сумується з поточним балансом, записується локально за допомогою SaveSystem та викликається метод SceneManager.LoadScene для повернення до меню.

Блок-схема алгоритму транзакції покупки та збереження покращень у гаражі:

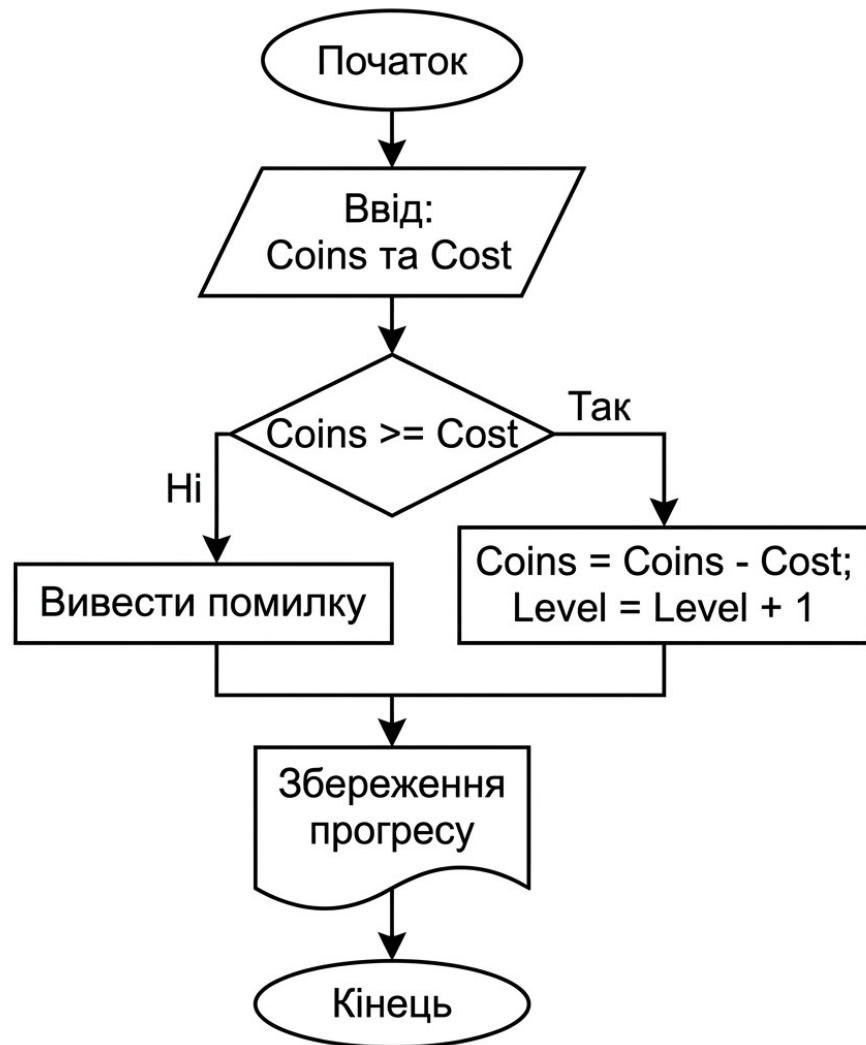


Рисунок 2.1 — Схема алгоритму перевірки монет та покупки покращень

—Контроль стану: Система постійно перевіряє температуру двигуна. При високих обертах температура підвищується. Досягнення максимального ліміту (120 градусів) призводить до перегріву та зупинки гри. Чекпоінти дозволяють охолодити двигун та отримати винагороду.—Збереження прогресу: При завершенні заїзду накопичені монети додаються до загального балансу гравця та серіалізуються локально у форматі JSON у файлі playerdata.json.Тестування програмного забезпечення є критично важливим етапом розробки мобільної гри «Bike Evolution», оскільки коректність роботи фізичної моделі руху, механіки трюків та системи збереження даних безпосередньо впливають на ігровий досвід користувача. Для перевірки працездатності проекту було застосовано такі методи:

— Модульне тестування (Unit Testing) — для ізольованої перевірки розрахунку вартості оновлень деталей двигуна та трансмісії за формулою в UpgradeUI, а також методів серіалізації JSON у класі OfflineSaveSystem.

— Інтеграційне тестування (Integration Testing) — для перевірки взаємодії між системою збереження та інтерфейсом гаражу. Тестувалися переходи між сценами та правильність передачі значень рівнів деталей з PlayerPrefs до BikeController при старті гонки.

— Ручне функціональне тестування (Manual Testing) — безпосередній запуск гри на емуляторах та мобільних пристроях для перевірки фізики їзди, зчеплення коліс з трасою, балансування при виконанні трюку та чутливості сенсорних кнопок керування.

У процесі тестування було виявлено кілька критичних помилок, які успішно виправлені на етапі розробки:

1. Проблема миттєвого перекидання мопеда: під час тестування кнопки трюку крутий момент був занадто великим, і переднє колесо різко піднімалося, через що водій одразу падав на спину. Для вирішення проблеми в скрипт BikeController було додано механізм Wheelie Assist, який зчитує поточний кут нахилу та обмежує прикладений момент при досягненні 45 градусів.

2. Збій збереження монет при аварійному виході: якщо користувач закривав гру під час заїзду за допомогою системного меню Android, отримані монети втрачалися. Проблема вирішено шляхом виклику примусового збереження даних за допомогою методу PlayerPrefs.Save() безпосередньо під час перетину чекпоінтів та при падінні водія.

3. Відсутність охолодження двигуна на холостому ході: температура двигуна не знижувалася після повної зупинки мопеда. Проблема вирішено додаванням умови в FixedUpdate, яка зменшує показник температури на значення ambientCoolingRate, коли throttleInput дорівнює нулю.

Інструкція з встановлення та запуску мобільної гри «Bike Evolution»:

— Встановлення на мобільний пристрій: скопіюйте файл «Bike Evolution.apk» на ваш Android-пристрій. У налаштуваннях безпеки пристрою дозвольте встановлення додатків з невідомих джерел. Запустіть інсталяційний файл та дочекайтеся завершення встановлення.

— Запуск у Unity Editor: відкрийте проект у середовищі розробки Unity Hub. Завантажте сцену «Login 1» (Assets/_Project/Scenes/Login 1.unity) та натисніть кнопку «Play» у верхній частині інтерфейсу редактора.

Покрокова інструкція користувача:

1. Після запуску гри відображається Головне меню. Тут є три основні кнопки: «Грати» (запуск заїзду), «Гараж» (меню покращення деталей мопеда) та «Вихід».

2. Перейдіть до вікна «Гараж». У верхньому кутку відображається поточна кількість накопичених монет. Для покращення доступні дві деталі: «Двигун» (двигун) та «Зірки» (зірочки приводу). Натискання відповідної кнопки купує покращення, підвищує рівень деталі та автоматично списує монети.

3. Поверніться до меню та натисніть «Грати». На екрані заїзду у правому нижньому кутку розміщено кнопку газу, у лівому нижньому кутку — кнопки переднього гальма, заднього гальма та кнопку «Stunt» (трюк).

4. Керування мопедом здійснюється натисканням та утриманням кнопок. Натискання кнопки газу розганяє заднє колесо. Утримання кнопки трюку піднімає мопед на заднє колесо. Слідкуйте за кутом нахилу мопеда на панелі, щоб уникнути падіння водія.

У ході виконання курсової роботи було успішно спроектовано, розроблено та протестовано мобільну 2D-гру «Bike Evolution», присвячену їзді та тюнінгу мопедів. Усі завдання, поставлені на етапі планування проекту,

були виконані у повному обсязі. Створений програмний продукт дозволяє користувачам відчувати атмосферу гаражного тюнінгу деталей та перевірити свої навички балансування техніки на задньому колесі.

В результаті написання курсової роботи було реалізовано такі ключові модулі:

1. Фізична модель руху двоколісного транспорту: на основі компонентів Rigidbody2D та WheelJoint2D розроблено реалістичну систему коліс, амортизації, аеродинамічного опору повітря та передачі кутового моменту на заднє колесо.

2. Система допомоги водієві (Wheelie Assist): інтегровано автоматичне обмеження крутного моменту двигуна при досягненні оптимального кута балансування (45 градусів) під час їзди на задньому колесі, що дозволяє гравцеві стабільно утримувати рівновагу.

3. Система гаражного тюнінгу деталей: розроблено uGUI-інтерфейс для покупки оновлень двигуна (підвищення потужності maxHP) та зірок трансмісії (корекція передавального числа), з автоматичним розрахунком прогресивної вартості рівнів.

4. Локальне збереження прогресу гравця: розроблено надійний клас OfflineSaveSystem, який автоматично серіалізує баланс монет та конфігурацію мопеда у файл playerdata.json на мобільному пристрої.

5. Модель терморегуляції двигуна: реалізовано механіку нагрівання двигуна під час тривалого утримання газу та його охолодження при перетині чекпоінтів або русі накатом.

Під час виконання цієї курсової роботи я суттєво поглибив знання мови програмування C# та об'єктно-орієнтованого програмування, отримав практичний досвід розробки мобільних інтерфейсів та оптимізації фізичного обчислювального циклу FixedUpdate у рушії Unity. Створена гра є повністю завершеним і працездатним прототипом, який готовий до подальшого релізу

та розширення новими трасами, моделями мопедів та таблицею лідерів онлайн.

1. Документація Unity API Reference. [Електронний ресурс] — Режим доступу: <https://docs.unity3d.com/ScriptReference/>
2. Піхтер Д. Програмування на платформі Microsoft .NET Framework 4.5 мовою С#. — Спб.: Питер, 2013. — 720 с.
3. Паттерни проектування в Unity. [Електронний ресурс] — Режим доступу: <https://learn.unity.com/>
4. Шилдт Г. С# 4.0: Повне керівництво. — М.: Вільямс, 2011. — 1056 с.
5. Бонд Д. Рецепти програмування на С# для розробки ігор в Unity. — Спб.: Питер, 2019. — 480 с.
6. Документація Microsoft .NET C# Language Guide. [Електронний ресурс] — Режим доступу: <https://docs.microsoft.com/en-us/dotnet/csharp/>
7. Фрімен А. Паттерни проектування. — М.: Вільямс, 2016. — 650 с.
8. Торн А. Розробка 2D ігор в Unity. — М.: ДМК Прес, 2015. — 350 с.

ДОДАТОК А. ПОРІВНЯЛЬНА ХАРАКТЕРИСТИКА ПРОГРАМНИХ ПРОДУКТІВ АНАЛОГІЧНОГО ПРИЗНАЧЕННЯ

У цьому додатку наведено порівняльний аналіз розробленої мобільної гри «Bike Evolution» з існуючими комерційними аналогами на ринку мобільних розваг: «Hill Climb Racing» та «Moto X3M».

Критерій порівняння	Hill Climb Racing	Moto X3M	Bike Evolution (проект)
Жанр	2D	2D	2D
/ Вимір	аркадний симулятор	трюкова аркада	фізичний симулятор

тюнінгом			
Фізи	Спрощена	Спрощена	Реалістич
чна модель	аркадна фізика	трюкова фізика	на двоколісна модель з аеродинамікою
Сист	Багато	Відсутня	Двигун
ема тюнінгу	деталей (двигун, підвіска, колеса)	(лише скіни)	(maxHP) та зірки трансмисії
Темп	Відсутній	Відсутній	Температ
ературний			урний режим та
режим			перегрів при
двигуна			високих RPM
Збер	Хмарне	Локальне	Локальні
еження	(Google Play	сховище	й файл JSON
прогресу	Games)	браузера	(playerdata.json)
Офла	Частково	Частково	Повний
йн-режим	(потребує мережі для реклами)		автономний режим (без реклами)

ДОДАТОК Б. UML ДІАГРАМА ПРЕЦЕДЕНТІВ

Діаграма прецедентів (Use Case Diagram) відображає основні варіанти використання системи користувачем (гравцем):

—Грати: запуск сцени заїзду, керування рухом мопеда, балансування на задньому колесі та виконання трюку;—Керування тюнінгом: перехід у меню Гаражу, вибір деталей та покупка покращень;—Перегляд характеристик: відображення балансу монет та рівнів деталей;—Автоматичне збереження: запис оновлених монет та конфігурації у локальний JSON-файл.*[картинка: UML діаграма прецедентів мобільної гри "Bike Evolution"]*

ДОДАТОК В. ПРОТОТИП ІНТЕРФЕЙСУ

У цьому додатку наведено ескізи прототипів інтерфейсу користувача, створені на етапі проектування для узгодження з вимогами:

—Головне меню: містить логотип гри, кнопку гонки «Грати», кнопку переходу до вікна тюнінгу «Гараж» та кнопку «Вихід»;—Меню Гаражу: візуалізація мопеда в центрі, поле балансу монет у кутку, дві кнопки покупки покращень з ціною;—Ігровий екран: кнопки газу, гальм та трюку «Stunt», показники швидкості та температури двигуна.*[картинка: Прототип інтерфейсу Головного меню]*

[картинка: Прототип інтерфейсу вікна Гаражу та тюнінгу]

[картинка: Прототип інтерфейсу ігрового екрану заїзду]

ДОДАТОК Г. UML ДІАГРАМА КЛАСІВ

Діаграма класів (Class Diagram) відображає об'єктно-орієнтовану структуру проекту, класи C# та зв'язки між ними:

—BikeController: основний клас фізики руху та моделювання обертів і нагріву двигуна;—UpgradeUI: клас відображення інтерфейсу гаражу та обробки покупки покращень;—OfflineSaveSystem: сервіс локального збереження даних гравця у формат JSON;—SaveData: структура збережених даних гравця.*[картинка: UML діаграма класів проекту "Bike Evolution"]*

ДОДАТОК Д. UML ДІАГРАМА РОЗГОРТАННЯ

Діаграма розгортання (Deployment Diagram) відображає фізичний розподіл компонентів на пристрої Android:

—Мобільний пристрій (Android Device): апаратна платформа;—
 Операційна система Android OS: системне серцевище;—Додаток Bike
 Evolution APK: виконуваний ігровий пакет;—Файл збереження
 playerdata.json: локальний файл даних у persistentDataPath.*[картинка: UML
 діаграма розгортання мобільного застосунку]*

ДОДАТОК Е. ВИХІДНІ КОДИ

У цьому додатку наведено повний вихідний код класів OfflineSaveSystem.cs та UpgradeUI.cs, а також ключові методи фізичного розрахунку класу BikeController.cs.

1. Вихідний код класу OfflineSaveSystem.cs

```
using System.IO;
using System.Threading.Tasks;
using UnityEngine;

public class OfflineSaveSystem : ISaveRepository
{
    private readonly string filePath;
    public OfflineSaveSystem(string fileName = "playerdata.json")
    {
        this.filePath = System.IO.Path.Combine(Application.persistentDataPath, fileName);
    }

    public void Save<T>(T data)
    {
        string json = JsonUtility.ToJson(data, true);
        System.IO.File.WriteAllText(filePath, json);
        Debug.Log(json);
    }

    public T Load<T>() where T : new()
    {
        if (!File.Exists(filePath))
            return new T();
        string json = File.ReadAllText(filePath);
        return JsonUtility.FromJson<T>(json);
    }

    public void DeleteSave()
    {
        if (File.Exists(filePath))
        {
            File.Delete(filePath);
        }
    }
}
```

```
}
```

2. Вихідний код класу UpgradeUI.cs

```
using UnityEngine;
using UnityEngine.UI;
using TMPro;

public class UpgradeUI : MonoBehaviour
{
    public TextMeshProUGUI coinsText;

    [Header("Engine Upgrade")]
    public TextMeshProUGUI engineLevelText;
    public TextMeshProUGUI engineCostText;
    public Button engineUpgradeButton;

    [Header("Primary Drive Upgrade")]
    public TextMeshProUGUI primaryDriveLevelText;
    public TextMeshProUGUI primaryDriveCostText;
    public Button primaryDriveUpgradeButton;

    private void Start()
    {
        if (engineUpgradeButton != null)
        {
            engineUpgradeButton.onClick.AddListener(UpgradeEngine);
        }
        if (primaryDriveUpgradeButton != null)
        {
            primaryDriveUpgradeButton.onClick.AddListener(UpgradePrimaryDrive);
        }
        UpdateUI();
    }

    private void UpdateUI()
    {
        int coins = PlayerPrefs.GetInt("Coins", 0);
        int engineLevel = PlayerPrefs.GetInt("EngineLevel", 1);
        int primaryDriveLevel = PlayerPrefs.GetInt("PrimaryDriveLevel", 1);

        int engineCost = GetUpgradeCost(engineLevel);
        int primaryDriveCost = GetUpgradeCost(primaryDriveLevel);

        if (coinsText != null)
        {
            coinsText.text = FormatValue(coins);
        }

        if (engineLevelText != null)
        {
            engineLevelText.text = "Lvl " + engineLevel;
        }
    }
}
```

```

if (engineCostText != null)
{
    engineCostText.text = FormatValue(engineCost);
}

if (primaryDriveLevelText != null)
{
    primaryDriveLevelText.text = "Lvl " + primaryDriveLevel;
}

if (primaryDriveCostText != null)
{
    primaryDriveCostText.text = FormatValue(primaryDriveCost);
}

if (engineUpgradeButton != null)
{
    engineUpgradeButton.interactable = coins >= engineCost;
}

if (primaryDriveUpgradeButton != null)
{
    primaryDriveUpgradeButton.interactable = coins >= primaryDriveCost;
}
}

private int GetUpgradeCost(int level)
{
    return 2 * level * level + 16 * level + 32;
}

private void UpgradeEngine()
{
    int coins = PlayerPrefs.GetInt("Coins", 0);
    int engineLevel = PlayerPrefs.GetInt("EngineLevel", 1);
    int cost = GetUpgradeCost(engineLevel);

    if (coins >= cost)
    {
        PlayerPrefs.SetInt("Coins", coins - cost);
        PlayerPrefs.SetInt("EngineLevel", engineLevel + 1);
        PlayerPrefs.Save();

        ApplyUpgradesToBike();
        UpdateUI();

        if (VirtualBalanceUI.Instance != null)
        {
            VirtualBalanceUI.Instance.RefreshBalance();
        }
    }
}

private void UpgradePrimaryDrive()
{
    int coins = PlayerPrefs.GetInt("Coins", 0);
    int primaryDriveLevel = PlayerPrefs.GetInt("PrimaryDriveLevel", 1);
    int cost = GetUpgradeCost(primaryDriveLevel);

    if (coins >= cost)

```

```

    {
        PlayerPrefs.SetInt("Coins", coins - cost);
        PlayerPrefs.SetInt("PrimaryDriveLevel", primaryDriveLevel + 1);
        PlayerPrefs.Save();

        ApplyUpgradesToBike();
        UpdateUI();

        if (VirtualBalanceUI.Instance != null)
        {
            VirtualBalanceUI.Instance.RefreshBalance();
        }
    }
}

private void ApplyUpgradesToBike()
{
    var raceService = FindFirstObjectByType<RaceService>();
    if (raceService != null)
    {
        var bikeController = FindFirstObjectByType<BikeController>();
        if (bikeController != null)
        {
            raceService.ApplyUpgrades(bikeController);
        }
    }
}

private string FormatValue(float value)
{
    if (value >= 1000000)
    {
        return (value / 1000000f).ToString("0.##") + "M";
    }
    if (value >= 1000)
    {
        return (value / 1000f).ToString("0.##") + "k";
    }
    return value.ToString();
}
}

```

3. Вихідний код класу BikeController.cs (вибрані методи)

```

using System.Collections.Generic;
using UnityEngine;

public class BikeController : MonoBehaviour
{
    [Header("Данные Двигателя")]
    public AnimationCurve activeEngineCurve;
    public float maxRPM = 8500f;
    public float maxHP = 4.5f;
    public float idleRPM = 1500f;
    public float engineBrakingFactor = 2.0f;

    [Header("Сцепление и Статус")]
    public bool isEngineRunning = true;
    public bool isEngineBroken = false;
    public float stallRPM = 1000f;
    [Range(0f, 1f)] public float clutchLock = 0f;
}

```

```

[Header("Трансмиссия (АКПП)")]
public float primaryDriveRatio = 3.72f;
public int frontSprocket = 14;
public int rearSprocket = 41;

public float[] gearRatios = { 0f, 3.27f, 1.93f, 1.35f, 1.04f };
public int currentGear = 0;

public float shiftUpRPM = 7500f;
public float shiftDownRPM = 3000f;
public float shiftDelay = 0.3f;
private float shiftTimer = 0f;

public float shiftCooldown = 1.0f;
private float currentCooldown = 0f;

public bool manualTransmission = false;

[Header("Стант: Физика и Колеса")]
public Rigidbody2D frameRb;
public Rigidbody2D rearWheelRb;
public Rigidbody2D frontWheelRb;

public Vector2 centerOfMassOffset = new Vector2(-0.1f, -0.3f);
public float wheelRadius = 0.28f;
public float arcadeTorqueMultiplier = 1.5f;

[Header("Аэродинамика")]
public float airResistance = 0.5f;
public float rollingResistance = 0.1f;

[Header("Стант: Ассистент Вилли")]
public bool enableWheelieAssist = true;
public bool enableEasyStuntMode = false;
public bool enableAutoThrottleFeathering = true;
public bool enableAdaptiveThrottleSpeed = true;
public bool enableSmoothStuntPull = true;
public bool enableGearBasedStuntScaling = false;
public float balanceAngle = 45f;
public float criticalAngle = 65f;

[Header("Управление (Ввод)")]
public MobileButton gasButton;
public MobileButton rearBrakeButton;
public MobileButton frontBrakeButton;
public MobileButton stuntButton;
public MobileButton shiftUpButton;
public MobileButton shiftDownButton;
public float stuntPullTorque = 150f;

[Range(0f, 1f)] public float throttleInput = 0f;
public float throttleSpeed = 5f;
public float stuntThrottleSpeed = 15f;
public float mechanicalBrakePower = 30f;

[Header("Состояние деталей (Износ)")]
[Range(0f, 1f)] public float engineCondition = 1f;
[Range(0f, 1f)] public float wheelsCondition = 1f;

[Header("Температура двигателя (Enduro)")]

```

```

public float engineTemperature = 20f;
public float maxEngineTemperature = 120f;
public float baseHeatRate = 10f;
public float ambientCoolingRate = 5f;
public float engineHeatMultiplier = 1.0f;
private const float OVERHEAT_RPM_THRESHOLD = 0.85f;
public bool isOverheated = false;
public bool enableTemperature = false;

public System.Action onOverheat;

[Header("Дебар")]
public float currentEngineRPM;
public float currentSpeedKmh;
public float currentPitchAngle;

private bool InputGas
{
    get
    {
        bool pressed = gasButton != null && gasButton.IsPressed;
#ifdef UNITY_EDITOR
        if (UnityEngine.InputSystem.Keyboard.current != null)
        {
            pressed |= UnityEngine.InputSystem.Keyboard.current.wKey.isPressed ||
UnityEngine.InputSystem.Keyboard.current.upArrowKey.isPressed;
        }
#endif
        return pressed;
    }
}

private bool InputStunt
{
    get
    {
        bool pressed = stuntButton != null && stuntButton.IsPressed;
#ifdef UNITY_EDITOR
        if (UnityEngine.InputSystem.Keyboard.current != null)
        {
            pressed |= UnityEngine.InputSystem.Keyboard.current.spaceKey.isPressed;
        }
#endif
        // ... [інші змінні та методи класу приховано для відображення ключової логіки] ...
    }
}

private void FixedUpdate()
{
    if (frameRb != null)
    {
        currentPitchAngle = frameRb.transform.eulerAngles.z;
        if (currentPitchAngle > 180f) currentPitchAngle -= 360f;
    }
    if (shiftTimer > 0) shiftTimer -= Time.fixedDeltaTime;
    if (currentCooldown > 0) currentCooldown -= Time.fixedDeltaTime;

    CalculateSpeedAndRPM();
    HandleAutomaticTransmission();
    ApplyEnginePhysics();
    ApplyMechanicalBrakes();
    ApplyAerodynamics();
    UpdateEngineTemperature();
}

private void ApplyAerodynamics()
{

```

```

float speed = frameRb.linearVelocity.magnitude;

if (speed < 0.1f) return;

float dragForceMagnitude = speed * speed * airResistance;
float rollingForceMagnitude = speed * rollingResistance;

Vector2 dragForce = -frameRb.linearVelocity.normalized * (dragForceMagnitude + rollingForceMagnitude);

frameRb.AddForce(dragForce);
}

// ... [код приховано для оптимізації обсягу звіту] ...
private void ApplyEnginePhysics()
{
    if (!isEngineRunning || shiftTimer > 0 || currentGear == 0) return;

    float wheelRevsPerSecond = Mathf.Abs(rearWheelRb.angularVelocity) / 360f;
    float wheelRPM = wheelRevsPerSecond * 60f;
    float mechanicalRPM = wheelRPM * CurrentTotalRatio;

    float normalizedRPM = currentEngineRPM / maxRPM;
    float powerPercentage = activeEngineCurve.Evaluate(normalizedRPM);
    float currentHP = powerPercentage * maxHP * Mathf.Max(engineCondition, 0.01f);

    float engineTorque = (currentHP * 7120f) / Mathf.Max(currentEngineRPM, 1f);
    float torqueAtWheel = engineTorque * CurrentTotalRatio * clutchLock * arcadeTorqueMultiplier *
wheelsCondition;

    if (mechanicalRPM >= maxRPM)
    {
        torqueAtWheel = 0f;
    }

    if (throttleInput > 0.05f)
    {
        float finalTorque = torqueAtWheel * throttleInput;

        float currentAngle = frameRb.transform.eulerAngles.z;
        if (currentAngle > 180f) currentAngle -= 360f;

        float gearMultiplier = 1f;
        if (enableGearBasedStuntScaling && currentGear > 1 && currentGear < gearRatios.Length)
        {
            gearMultiplier = Mathf.Sqrt(gearRatios[currentGear] / gearRatios[1]);
        }

        float actualStuntTorque = stuntPullTorque * gearMultiplier;

        if (InputStunt && frameRb != null)
        {
            if (enableEasyStuntMode)
            {
                float t = Mathf.InverseLerp(5f, balanceAngle, currentAngle);
                float currentStuntPull = Mathf.Lerp(actualStuntTorque, actualStuntTorque * 0.05f, t);

                if (frameRb.angularVelocity > 1.5f && currentAngle > 15f)
                {
                    frameRb.angularVelocity = Mathf.Lerp(frameRb.angularVelocity, 1.5f, 5f * Time.fixedDeltaTime);
                }
            }
        }
    }
}

```



```

        if (currentAngle < balanceAngle) frameRb.AddTorque(currentStuntPull);
    }
    else if (enableWheelieAssist)
    {
        float currentStuntPull = actualStuntTorque;

        if (enableSmoothStuntPull)
        {
            float smoothStart = balanceAngle - 15f;
            if (currentAngle > smoothStart && currentAngle < balanceAngle)
            {
                float t = Mathf.InverseLerp(smoothStart, balanceAngle, currentAngle);
                currentStuntPull = Mathf.Lerp(actualStuntTorque, actualStuntTorque * 0.1f, t);
            }
        }

        if (currentAngle < balanceAngle) frameRb.AddTorque(currentStuntPull);
    }
    else
    {
        frameRb.AddTorque(actualStuntTorque);
    }
}

if (!enableEasyStuntMode && enableWheelieAssist && frameRb != null && currentAngle > balanceAngle)
{
    float reductionFactor = 1f - Mathf.InverseLerp(balanceAngle, criticalAngle, currentAngle);
    finalTorque *= reductionFactor;

    if (InputStunt) frameRb.AddTorque(-actualStuntTorque * 0.5f);
}

if (finalTorque > 0f)
{
    rearWheelRb.AddTorque(-finalTorque);
}
else if (clutchLock > 0.5f)
{
    float brakingForce = engineBrakingFactor * CurrentTotalRatio;
    rearWheelRb.AddTorque(-Mathf.Sign(rearWheelRb.angularVelocity) * brakingForce);
}
}
}

```

ДОДАТОК Ж. РЕЗУЛЬТАТИ ТЕСТУВАННЯ

У цьому додатку представлено матрицю тест-кейсів, виконаних під час комплексного тестування програмного продукту «Bike Evolution».

ID

Опис тесту

Очікуваний результат

Статус

TC-01

Запуск гри без файлу збереження (перший старт).

Створюється новий профайл, баланс монет = 0, рівень деталей = 1.

Успішно

TC-02

Купівля оновлення двигуна в гаражі при Coins $\geq$ Cost.

Баланс зменшується, EngineLevel збільшується, потужність maxHP зростає.

Успішно

TC-03

Спроба купити оновлення в гаражі при Coins $<$ Cost.

Кнопка оновлення стає неактивною, транзакція покупки блокується.

Успішно

TC-04

Робота системи збереження Save() при виході в меню.

Нові значення записуються у файл playerdata.json, який зчитується без помилок.

Успішно

TC-05

Активація трюку (Stunt) під час руху мопеда.

Рама мопеда нахиляється назад, система Wheelie Assist стабілізує кут біля 45°.

Успішно

TC-06

Падіння водія головою на землю (CrashDetector).

Спрацьовує OnCrash(), викликається смерть водія, зменшується віртуальне здоров'я.

Успішно

TC-07

Перегрів двигуна до температури 120°C.

Двигун вимикається (isEngineRunning=false), з'являється вікно програшу.

Успішно

TC-08

Отримання чекпоінту (OnCheckpointReached).

Баланс збільшується, двигун охолоджується, точка респауну оновлюється.

Успішно

ДОДАТОК И. ЗНІМКИ ЕКРАНУ

У цьому додатку наведено фактичні знімки екрану працюючого додатку мобільної гри «Bike Evolution» на Android-пристрої:

[картинка: Знімок екрану головного меню гри]

[картинка: Знімок екрану гаражу та меню тюнінгу деталей]

[картинка: Знімок екрану ігрової сцени під час руху мопеда]

[картинка: Знімок екрану перегріву двигуна (вікно Overheat)]

[картинка: Знімок екрану аварії водія (вікно Crash)]

